

DBMS into iOS development:

1. Database Management: SQLite

SQLite is a lightweight, self-contained SQL database engine widely used in mobile applications.

Using SQLite in iOS

You can use SQLite directly in your iOS app or via a wrapper like **SQLite.swift**.

Example: Direct SQLite Usage

1. Add SQLite framework to your project.
2. Create and interact with the database.

```
import SQLite3
```

```
var db: OpaquePointer?
```

```
// Create or open the database
```

```
let fileURL = try! FileManager.default
```

```
    .url(for: .documentDirectory, in: .userDomainMask, appropriateFor: nil, create: false)
```

```
    .appendingPathComponent("Example.sqlite")
```

```
if sqlite3_open(fileURL.path, &db) != SQLITE_OK {
```

```
    print("Error opening database")
```

```
}
```

```
// Create a table
```

```
let createTableQuery = "CREATE TABLE IF NOT EXISTS Users (id INTEGER PRIMARY KEY  
AUTOINCREMENT, name TEXT, age INTEGER)"
```

```
if sqlite3_exec(db, createTableQuery, nil, nil, nil) != SQLITE_OK {
```

```
    print("Error creating table")
```

```
}
```

```
// Insert data
```

```
let insertQuery = "INSERT INTO Users (name, age) VALUES ('Alice', 25)"
```

```
if sqlite3_exec(db, insertQuery, nil, nil, nil) != SQLITE_OK {
```

```
    print("Error inserting data")
```

```
}
```

```
// Close database
```

```
sqlite3_close(db)
```

2. Web Services

Web services enable apps to communicate with remote servers using protocols like REST or SOAP.

Example: REST API with URLSession

```
let url = URL(string: "https://api.example.com/users")!
```

```
var request = URLRequest(url: url)
request.httpMethod = "GET"
```

```
let task = URLSession.shared.dataTask(with: request) { data, response, error in
    if let error = error {
        print("Error: \(error)")
        return
    }

    if let data = data {
        print(String(data: data, encoding: .utf8)!)
    }
}
task.resume()
```

3. JSON Parsing

JSON (JavaScript Object Notation) is a lightweight format for data exchange.

Parsing JSON in Swift

Given this JSON response:

```
{
  "name": "Alice",
  "age": 25
}
```

Example

```
struct User: Codable {
    let name: String
    let age: Int
}
```

```
let jsonData = """
```

```
{
  "name": "Alice",
  "age": 25
}
```

```
""".data(using: .utf8)!
```

```
do {
    let user = try JSONDecoder().decode(User.self, from: jsonData)
    print("Name: \(user.name), Age: \(user.age)")
} catch {
```

```

    print("Error decoding JSON: \(error)")
}

```

4. XML Parsing

XML parsing is less common than JSON in modern APIs but still used in some systems.

Example: XML Parsing with XMLParser

```
import Foundation
```

```

class XMLHandler: NSObject, XMLParserDelegate {
    var elementName: String = ""
    var currentValue: String = ""

    func parseXML(data: Data) {
        let parser = XMLParser(data: data)
        parser.delegate = self
        parser.parse()
    }

    func parser(_ parser: XMLParser, didStartElement elementName: String, namespaceURI:
String?, qualifiedName: String?, attributes attributeDict: [String : String] = [:]) {
        self.elementName = elementName
    }

    func parser(_ parser: XMLParser, foundCharacters string: String) {
        self.currentValue += string
    }

    func parser(_ parser: XMLParser, didEndElement elementName: String, namespaceURI:
String?, qualifiedName: String?) {
        if elementName == "name" {
            print("Name: \(currentValue)")
        }
        currentValue = ""
    }
}

```

5. Alamofire

Alamofire is a powerful HTTP networking library for Swift, simplifying network requests.

Example: Fetching Data

```
import Alamofire
```

```

Alamofire.request("https://api.example.com/users").responseJSON { response in
    switch response.result {
    case .success(let data):
        print("Data: \(data)")
    }
}

```

```

    case .failure(let error):
        print("Error: \(error)")
    }
}

```

Example: Post Request with Parameters

```

let parameters: [String: Any] = [
    "name": "Alice",
    "age": 25
]

```

```

Alamofire.request("https://api.example.com/users", method: .post, parameters:
parameters, encoding: JSONEncoding.default)
    .responseJSON { response in
        switch response.result {
        case .success(let data):
            print("Data: \(data)")
        case .failure(let error):
            print("Error: \(error)")
        }
    }
}

```

These examples provide a starting point for each topic. Let me know if you'd like further clarification or additional examples!